Лабораторная работа №3
Работа с прерываниями. NVIC. EXTI

Цель работы: изучить основу прерываний и научиться работать с прерываниями EXTI микроконтроллера STM32.

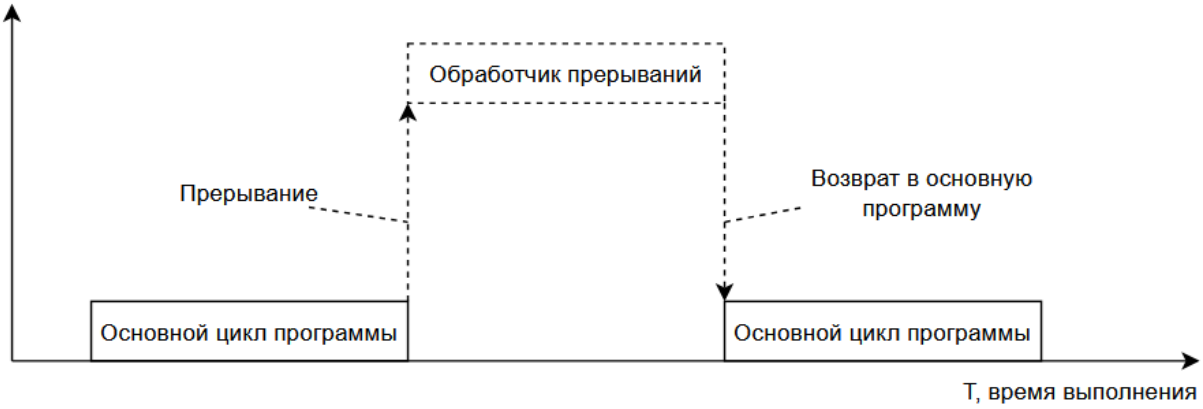
Общие сведения:

Interrupt – это программное прерывание. При прерывании обычно программа пакует регистры в стек и переходит по вектору, а оттуда через JMP перепрыгивает в обработчик прерывания.

В Cortex, вектора прерывания – это вектор-адреса, лежащие в нужном месте, в виде некоторых констант. При прерывании, программа не прыгает на вектор, а берет этот вектор-адрес и сразу же пихает его в программный счетчик, тем самым переходя сразу на обработчик. Так прерывание выполняется быстрей, поскольку исчезает лишняя операция по переходу на вектор.

Основная суть работы прерываний, если рассматривать их на более наглядном примере, следующая:

1) В какой-то момент выполнения основной программы, возникает событие, на которое нужно немедленно “отвлечься”
2) Управление программы переходит в обработчик события (прерывания), при этом сохраняя все переменные и вызовы функций из основного цикла в стеке
3) Прерывание обрабатывается (выполняется какой-то программный код)
4) По окончанию обработки, происходит переход обратно в основной цикл программы.



Event — это аппаратное событие, которое происходит при работе той или иной периферии. Например, опустел буфер UART — вызвался event, счетчик таймера дошел до конца — вызвался еще один event. Событие может вызвать прерывание, может запустить какую-либо периферию, например DMA, чтобы выгрузить данные из памяти. Но событие далеко не всегда может вызвать прерывание.

Каждое прерывание вызывается событием, но не каждое событие вызывает прерывание.

Обработка и управление прерываниями производятся аппаратным узлом микроконтроллера – контроллером приоритетных векторных прерываний (NVIC, Nested Vectored Interrupt Controller). При возникновении разрешенного события контроллер прерывает выполнение программы, сохраняет в стеке необходимые данные и передает управление по адресу функции обработки прерывания. После выхода из функции-обработчика управление возвращается на адрес, по которому была прервана основная программа.

С точки зрения программного управления, контроллер прерывания сам по себе является не сложным устройством. Из основных тезисов работы прерывания можно выделить следующие:

- Контроллер приоритетный. Т.е. обработка прерывания может быть прервана другим прерыванием с более высоким приоритетом. Прерывания с таким же или более низким приоритетом будут ожидать окончания обработки активного прерывания. Приоритеты можно разбивать на группы и подгруппы;
- Как следствие предыдущего пункта для каждого прерывания может быть задан приоритет. Это число от 0 до 15. 0 - самый высокий приоритет. Для установки приоритетов существует специальный блок регистров. На каждое прерывание в нем отведен один байт;
- Могут быть запрещены сразу все прерывания;
- Могут быть запрещены или разрешены отдельные прерывания. Для этого существует регистр разрешения или маски прерываний. Его отдельные биты определяют разрешение для каждого прерывания;
- Существует блок регистров ожидания обработки прерываний, каждый бит которых соответствует конкретному прерыванию и показывает, ожидает ли оно обработки. Биты регистров могут быть установлены, для принудительного перевода прерываний в состояние ожидания или сброшены для удаления.
- Есть регистры действующих прерываний, которые показывают, находятся ли прерывания в стадии обработки.
- Любое прерывание может быть вызвано программно. Для этого существует регистр программного вызова прерываний. Прерывание вызывается записью в него соответствующего номера.
- Каждому из прерываний соответствует свой фиксированный адрес-вектор, по которому передается управление для обработки прерывания.
- Формально обработкой и управлением прерываниями занимается контроллер прерываний. Но для возникновения прерывания необходимо выполнение нескольких условий связанных как с контроллером прерываний, так и с периферийными устройствами микроконтроллера.

Таблица векторов прерываний — это массив адресов, где каждый элемент таблицы содержит адрес функции-обработчика (ISR, Interrupt Service Routine) для конкретного прерывания. Когда происходит прерывание, процессор автоматически использует номер

прерывания как индекс в этой таблице, чтобы найти адрес соответствующего обработчика.

В микроконтроллерах STM32F103 поддерживается до 68 аппаратных прерываний. Каждому из них соответствует свой адрес – вектор. По этому адресу передается управление программы при возникновении прерывания.

Конкретные адреса для каждого источника можно посмотреть в документации на микроконтроллер. Выглядит это примерно так.

Table 63. Vector table for other STM32F10xxx devices (continued)

Position	Priority	Type of priority	Acronym	Description	Address
14	21	settable	DMA1_Channel4	DMA1 Channel4 global interrupt	0x0000_0078
15	22	settable	DMA1_Channel5	DMA1 Channel5 global interrupt	0x0000_007C
16	23	settable	DMA1_Channel6	DMA1 Channel6 global interrupt	0x0000_0080
17	24	settable	DMA1_Channel7	DMA1 Channel7 global interrupt	0x0000_0084
18	25	settable	ADC1_2	ADC1 and ADC2 global interrupt	0x0000_0088
19	26	settable	USB_HP_CAN_TX	USB High Priority or CAN TX interrupts	0x0000_008C
20	27	settable	USB_LP_CAN_RX0	USB Low Priority or CAN RX0 interrupts	0x0000_0090
21	28	settable	CAN_RX1	CAN RX1 interrupt	0x0000_0094
22	29	settable	CAN_SCE	CAN SCE interrupt	0x0000_0098
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000_009C
24	31	settable	TIM1_BRK	TIM1 Break interrupt	0x0000_00A0
25	32	settable	TIM1_UP	TIM1 Update interrupt	0x0000_00A4
26	33	settable	TIM1_TRG_COM	TIM1 Trigger and Commutation interrupts	0x0000_00A8
27	34	settable	TIM1_CC	TIM1 Capture Compare interrupt	0x0000_00AC
28	35	settable	TIM2	TIM2 global interrupt	0x0000_00B0
29	36	settable	TIM3	TIM3 global interrupt	0x0000_00B4
30	37	settable	TIM4	TIM4 global interrupt	0x0000_00B8

Стандартные характеристики NVIC:

- до 240 маскируемых прерываний (в зависимости от конкретной модели процессора)
- программируемый уровень приоритета 0-15 для каждого прерывания. Более высокий уровень соответствует более низкому приоритету. Уровень 0 соответствует прерыванию с наивысшим приоритетом
- обнаружение сигналов прерываний по фронтам и импульсного типа
- 1 немаскируемое прерывание (прерывание, которое нельзя отключить);
- динамическое изменение приоритетов прерываний
- группировка приоритета в приоритетную группу
- цепочки прерываний
- внешнее немаскируемое прерывание (NMI)

Нюансы при работе с прерываниями

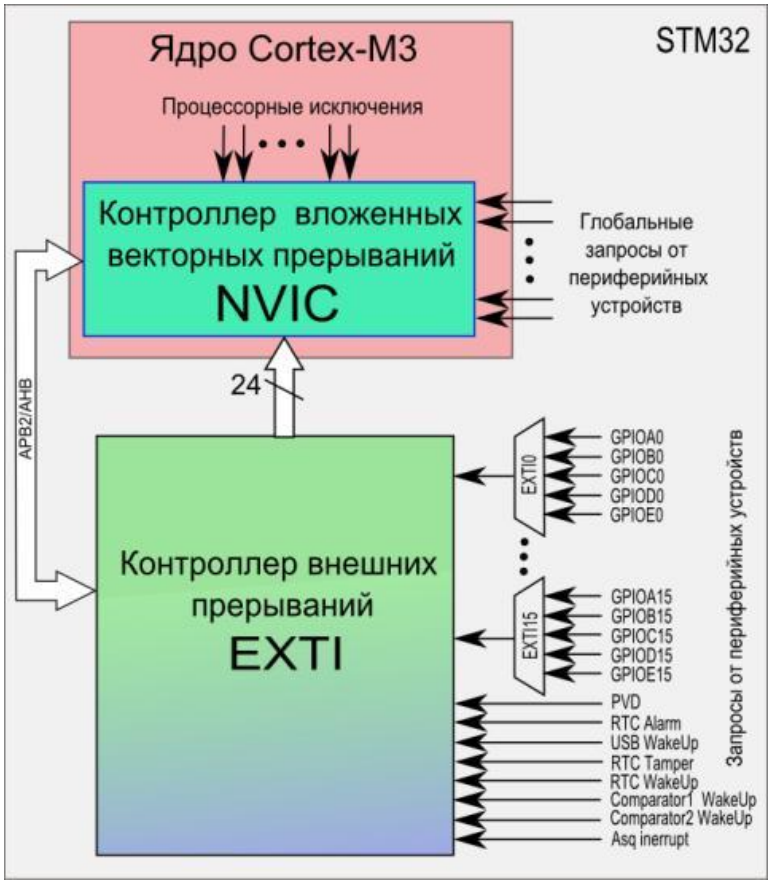
Как было ранее сказано, NVIC поддерживает прерывания с различными приоритетами, которые могут прерывать друг друга. При этом, могут возникнуть различные ситуации, обработка которых по разному оптимизирована.

1) Приостановка низкоприоритетного прерывания
   В этой ситуации, обработка низкоприоритетного прерывания прекращается. Следующие 12 циклов выполняется сохранение в стек нового набора данных и запускается обработка высокоприоритетного прерывания. После его обработки, содержимое стека автоматически извлекается и возобновляется обработка низкоприоритетного прерывания.
   Больших отличий от прерывания основной программы не наблюдается.

2) Непрерывная обработка прерываний
   Эта ситуация может возникнуть в двух случаях: если два прерывания имеют одинаковый приоритет и возникают одновременно, если низкоприоритетное прерывание возникает во время обработки высокоприоритетного.
   В этом случае, промежуточные операции над стеком не производятся. Происходит только загрузка адреса обработчика низкоприоритетного прерывания и переход к его выполнению. Отказ от операций над стеком экономит 6 тактов. Переход к следующему прерыванию происходит не за 12 тактов, а всего за 6.

3) Запаздывание высокприоритетного прерывания
   Ситуация возникает, если высокоприоритетное прерывание происходит во перехода к обработке низкоприоритетного (за те самые 12 тактов). В этом случае переход к высокоприоритетному прерыванию будет происходить не менее 6 тактов с момента его возникновения (время необходимое для загрузки адреса обработчика прерывания и перехода к нему). Возврат в низкоприоритетное уже описан выше.

Модуль вложенных векторных прерываний NVIC стал одним из нововведений ARM, при создании процессорного ядра Cortex. До этого задача обработки прерываний решалась блоками конкретных производителей микропроцессоров. С введением NVIC процедура обработки получила единообразие для разных процессоров, уменьшились требования к программному коду обработки исключительных ситуаций.



На уровне контроллера NVIC все прерывания делятся на два типа – прерывания от периферийных устройств или внешние и дополнительные или внутренние. К внутренним относятся разнообразные процессорные исключения, события системного таймера, и встроенного отладчика. Для разработки конечных приложений эти элементы используются редко, гораздо больший интерес представляют внешние прерывания. Данные сигналы поступают в контроллер NVIC по отдельным линиям, каждая из которых имеет свой номер и приоритет. Для расширения возможностей и повышения гибкости настроек часть периферийных устройств STM32 подключена не напрямую к NVIC, а к внешнему контроллеру EXTI.

EXTI (EXTernal Interrupt/Event)

Собственно, **EXTI (EXTernal Interrupt/Event)** – это контроллер внешних прерываний Внешние прерывания нужны для реакции прошивки МК на какие-либо быстро протекающие внешние события, которые проблематично регистрировать методом опроса состояния вывода GPIO. В STM32 для этих целей и нужен
EXTI, который как раз и ориентирован на обработку внешних прерываний.

Ниже представлены следующие особенности по работе с контроллером EXTI:

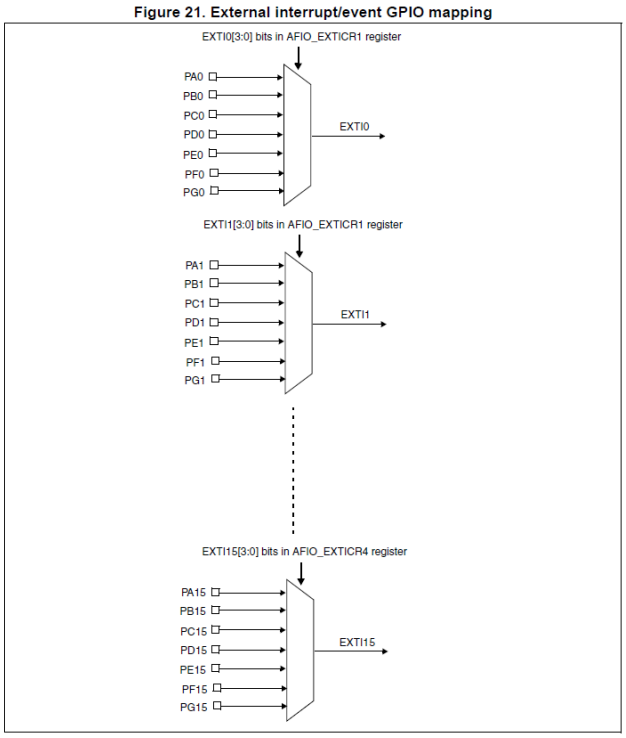
- Установка края импульса на входе канала EXTI, по которому будет генерироваться прерывание. Можно установить триггер по нарастающему краю импульса, по спадающему, или установить оба триггера сразу.
- Каждый канал имеет свой бит разрешения прерывания

- Каждый канал имеет свой бит ожидания запроса прерывания, который необходимо очистить в обработчике соответствующего прерывания
- Контроллер EXTI гарантированно обнаруживает импульсы, длительность которых больше длительность периода тактового сигнала шины APB2. Т.е. если частота шина APB2 равна 72 МГц, то EXTI будет корректно обнаруживать фронты сигналов с частотами ниже 72 МГц.

Каналы EXTI

Каналы EXTI имеют следующие названия: EXTI0, EXTI1, EXTI2 .. EXTI19. Всего в распоряжении 20 каналов. Причем EXTI0 — EXTI15 могут быть подключены к одному из портов GPIO. EXTI16 подключен внутри МК к выходу программируемого детектора напряжения PVD, EXTI17 к событию RTC Alarm, EXTI18 к USB микроконтроллера, и EXTI19 к контроллеру Ethernet, если он конечно есть.

В рамках данного занятия, нас интересуют те каналы EXTI, которые могут быть подключены к портам GPIO. И здесь есть один нюанс. На входе каждого канала EXTI стоит мультиплексор, который позволяет выбрать пин GPIO следующим образом:



Т.е. к EXTI0 можно подключить один из 0-ых выводов портов, к EXTI1 один из 1-ых выводов, и так далее. Для каждой линии значение мультиплексора можно выбрать независимо, т.е. EXTI0 можно подключить к PA0, EXTI1 к PB1, и так далее. Однако такая организация подключений имеет некоторое ограничение, которое необходимо учитывать: мы не можем одновременно регистрировать события от, например, линий PA0 и PB0, так как они подключены к одному и тому же мультиплексору. Иными словами, для EXTI с одним номером можно подключить либо PA0, либо PB0, но никак не два сразу!

Настройка работы прерываний

Для работы с аппаратными прерываниями необходимо:

1) Разрешить нужное прерывание в контроллере прерываний. Если необходимо, то задать для него приоритет.
2) Разрешить глобальные прерывания микроконтроллера.
3) Настроить устройство, в нашем случае таймер, на формирование запроса прерывания – события.
4) Создать функцию обработки прерывания.

Вообще, если касаться расположения прерываний на программном уровне, то их векторную таблицу можно найти в файле с расширением .s, где преставлены обработчики возможных прерываний.

```
; Vector Table Mapped to Address 0 at Reset

                AREA    RESET, DATA, READONLY
                EXPORT  __Vectors

__Vectors       DCD     __initial_sp              ; Top of Stack
                DCD     Reset_Handler             ; Reset Handler
                DCD     NMI_Handler               ; NMI Handler
                DCD     HardFault_Handler         ; Hard Fault Handler
                DCD     MemManage_Handler         ; MPU Fault Handler
                DCD     BusFault_Handler          ; Bus Fault Handler
                DCD     UsageFault_Handler        ; Usage Fault Handler
                DCD     0                         ; Reserved
                DCD     0                         ; Reserved
                DCD     0                         ; Reserved
                DCD     0                         ; Reserved
                DCD     SVC_Handler               ; SVCall Handler
                DCD     DebugMon_Handler          ; Debug Monitor Handler
                DCD     0                         ; Reserved
                DCD     PendSV_Handler            ; PendSV Handler
                DCD     SysTick_Handler           ; SysTick Handler

                ; External Interrupts
                DCD     WWDG_IRQHandler           ; Window Watchdog
                DCD     PVD_IRQHandler            ; PVD through EXTI Line detect
                DCD     TAMPER_IRQHandler         ; Tamper
                DCD     RTC_IRQHandler            ; RTC
                DCD     FLASH_IRQHandler          ; Flash
                DCD     RCC_IRQHandler            ; RCC
                DCD     EXTI0_IRQHandler          ; EXTI Line 0
                DCD     EXTI1_IRQHandler          ; EXTI Line 1
                DCD     EXTI2_IRQHandler          ; EXTI Line 2
                DCD     EXTI3_IRQHandler          ; EXTI Line 3
                DCD     EXTI4_IRQHandler          ; EXTI Line 4
                DCD     DMAChannel1_IRQHandler    ; DMA Channel 1
                DCD     DMAChannel2_IRQHandler    ; DMA Channel 2
                DCD     DMAChannel3_IRQHandler    ; DMA Channel 3
                DCD     DMAChannel4_IRQHandler    ; DMA Channel 4
                DCD     DMAChannel5_IRQHandler    ; DMA Channel 5
                DCD     DMAChannel6_IRQHandler    ; DMA Channel 6
                DCD     DMAChannel7_IRQHandler    ; DMA Channel 7
                DCD     ADC_IRQHandler            ; ADC
                DCD     USB_HP_CAN_TX_IRQHandler  ; USB High Priority or CAN TX
                DCD     USB_LP_CAN_RX0_IRQHandler ; USB Low  Priority or CAN RX0
                DCD     CAN_RX1_IRQHandler        ; CAN RX1
                DCD     CAN_SCE_IRQHandler        ; CAN SCE
                DCD     EXTI9_5_IRQHandler        ; EXTI Line 9..5
                DCD     TIM1_BRK_IRQHandler       ; TIM1 Break
                DCD     TIM1_UP_IRQHandler        ; TIM1 Update
                DCD     TIM1_TRG_COM_IRQHandler   ; TIM1 Trigger and Commutation
                DCD     TIM1_CC_IRQHandler        ; TIM1 Capture Compare
                DCD     TIM2_IRQHandler           ; TIM2
```

Однако, в большинстве случаев, рекомендуется придерживаться таблицы с описанием обработчиков и их адресов из даташита.

Программирование NVIC

Описание на NVIC можно найти в файле PM0056 STM32F10xxx Cortex-M3 programming manual. Там описывается только то, что есть в ядре Cortex M3. Можно конечно открыть более общее описание, взятое с сайта ARM. Но там будут небольшие отличия. Например, у STM32 урезана система группировки приоритетов.

Регистры NVIC

За работу прерываний отвечают несколько регистров, представленные в таблице на слайде

Table 41. Mapping of interrupts to the interrupt variables

Interrupts	CMSIS array elements (1)				
	Set-enable	Clear-enable	Set-pending	Clear-pending	Active Bit
0-31	ISER[0]	ICER[0]	ISPR[0]	ICPR[0]	IABR[0]
32-63	ISER[1]	ICER[1]	ISPR[1]	ICPR[1]	IABR[1]
64-67	ISER[2]	ICER[2]	ISPR[2]	ICPR[2]	IABR[2]

1. Each array element corresponds to a single NVIC register, for example the element ICER[1] corresponds to the ICER1 register.

- **ISER** — Interrupt Set Enable Register. Запись бита в нужную позицию включает прерывание.
- **ICER** — Interrupt Clr Enable Register. Запись сюда наоборот выключает прерывание.

Запись 1 в биты этих регистров запрещает/разрешает прерывания. Запись 0 не делает ничего, а чтение возвращает текущее состояние разрешено/запрещено

- **ISPR** — Interrupt Set Pending Register. Поставить прерывание в ожидание.
- **ICPR** — Interrupt Clr Pending Register. Сбросить прерывание с ожидания.

Запись 1 в биты этих регистров ставит/снимает прерывания в очередь на исполнение. Запись 0 не делает ничего, а чтение возвращает текущее состояние прерывания. Ждет он обработки или уже нет. Т.е. если в этом регистре где то стоит бит 1, значит это прерывание еще не вызывалось.

- **IABR** — Interrupt active bit registers. Регистр показывающий активно ли в данный момент прерывание. Автоматически ставится когда мы попадаем в обработчик и автоматом же снимается когда мы уходим из него. Этот регистр доступен только для чтения.

Функции HAL для работы с NVIC

В библиотеке HAL (Hardware Abstraction Layer) для микроконтроллеров STM32, работа с NVIC (Nested Vectored Interrupt Controller) осуществляется через набор функций, которые упрощают управление прерываниями. NVIC — это аппаратный модуль, который управляет приоритетами и маскированием прерываний в Cortex-M процессорах.

HAL_NVIC_EnableIRQ(IRQn_Type IRQn)

Включает прерывание по указанному номеру (IRQn).

Пример: HAL_NVIC_EnableIRQ(EXTI0_IRQn); // включает прерывание EXTI0.

HAL_NVIC_DisableIRQ(IRQn_Type IRQn)

Отключает прерывание по указанному номеру (IRQn).

Пример: HAL_NVIC_DisableIRQ(EXTI0_IRQn); //отключает прерывание EXTI0.

HAL_NVIC_SetPriority(IRQn_Type IRQn, uint32_t PreemptPriority, uint32_t SubPriority)

Устанавливает приоритет для указанного прерывания.

PreemptPriority — приоритет вытеснения (чем меньше значение, тем выше приоритет).

SubPriority — подприоритет (используется, если несколько прерываний имеют одинаковый приоритет вытеснения).

Пример: HAL_NVIC_SetPriority(EXTI0_IRQn, 1, 0); // устанавливает приоритет 1 и подприоритет 0 для EXTI0.

Регистры EXTI

Для настройки прерываний, фронтов срабатывания и так далее существуют регистры, расположенные в адресном пространстве контроллера EXTI. Мы рассмотрим все регистры EXTI которые относятся к настройке мультиплексоров EXTI.

EXTI_IMR

Interrupt mask register (EXTI_IMR) — Регистр маски прерываний

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												MR19	MR18	MR17	MR16
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

MRx. Разрешения прерывания канала x (0 - 19)

- 0 — прерывание отключено
- 1 — прерывание включено.

EXTI_RTSR

Rising trigger selection register (EXTI_RTSR) — Регистр включения детектора нарастающего края импульса

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												TR19	TR18	TR17	TR16
												rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

TRx. Разрешить срабатывание триггера нарастающего края импульса канала x

- 0 — триггер выключен
- 1 — триггер включен

EXTI_FTSR

Falling trigger selection register (EXTI_FTSR) — Регистр включения детектора спадающего края импульса

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												TR19	TR18	TR17	TR16
												rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

TRx – Разрешить срабатывание триггера спадающего края импульса канала x

- 0 — триггер выключен
- 1 — триггер включен

EXTI_SWIER

Software interrupt event register (EXTI_SWIER) — Регистр программного прерывания

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												SWIER 19	SWIER 18	SWIER 17	SWIER 16
												rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWIER 15	SWIER 14	SWIER 13	SWIER 12	SWIER 11	SWIER 10	SWIER 9	SWIER 8	SWIER 7	SWIER 6	SWIER 5	SWIER 4	SWIER 3	SWIER 2	SWIER 1	SWIER 0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

SWIERx – Программное прерывание линии x

Если прерывание данного канала EXTI включено в регистре EXTI_IMR, то запись ‘1’ в соответствующий бит, если до этого там было ‘0’, приводит к установке соответствующего бита в регистре EXTI_PR и генерируется запрос на прерывание данного канала EXTI.

Этот бит очищается при очистке соответствующего бита в регистре EXTI_PR.

EXTI_PR

Pending register (EXTI_PR) — Регистр ожидания

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												PR19	PR18	PR17	PR16
												rc_w1	rc_w1	rc_w1	rc_w1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PR15	PR14	PR13	PR12	PR11	PR10	PR9	PR8	PR7	PR6	PR5	PR4	PR3	PR2	PR1	PR0
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

PRx: Бит ожидания обработки прерывания канала x

- 0 — события триггеров запроса на прерывания не произошли
- 1 — произошли события триггеров запроса на прерывание

Если соответствующее прерывание разрешено в контроллере прерываний NVIC, то установка бита в регистре EXTI_PR приведет к вызову соответствующего обработчика прерывания. После вызова обработчика соответствующий бит не сбрасывается автоматически, это нужно сделать вручную, иначе сразу после выхода из обработчика прерывания он будет вызван снова, и так до бесконечности.

Сброс бита в регистре EXTI_PR выполняется путем записи в соответствующий бит значения ‘1’. Запись значения ‘0’ не оказывает ни какого эффекта.

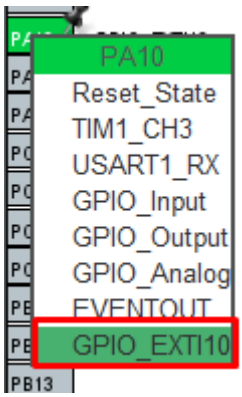
Функция обработки прерываний EXTI

Для обработки прерываний EXTI GPIO, используется преимущественно одна функция: **HAL_GPIO_EXTI_Callback**

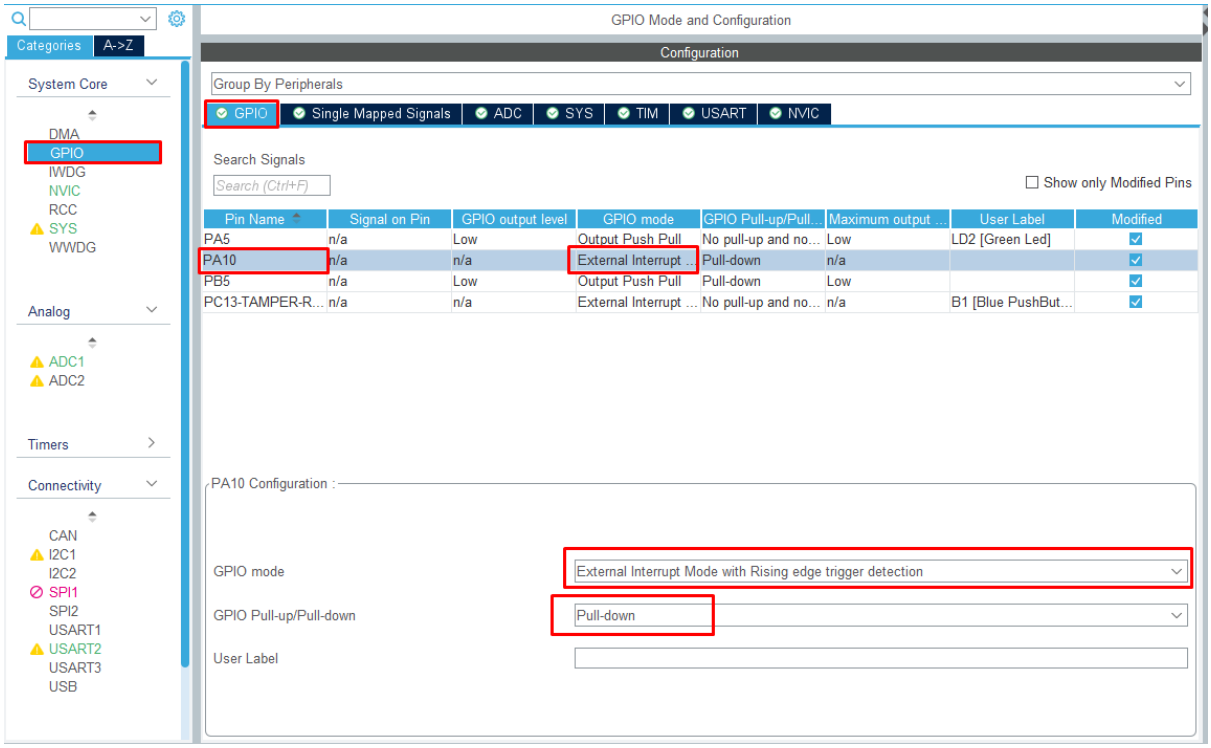
```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
	if(GPIO_Pin == GPIO_PIN_10)
	{
		/* выполнить какое-то действие */
	}
}
```

Включение EXTI-прерывания в STM32CubeIDE

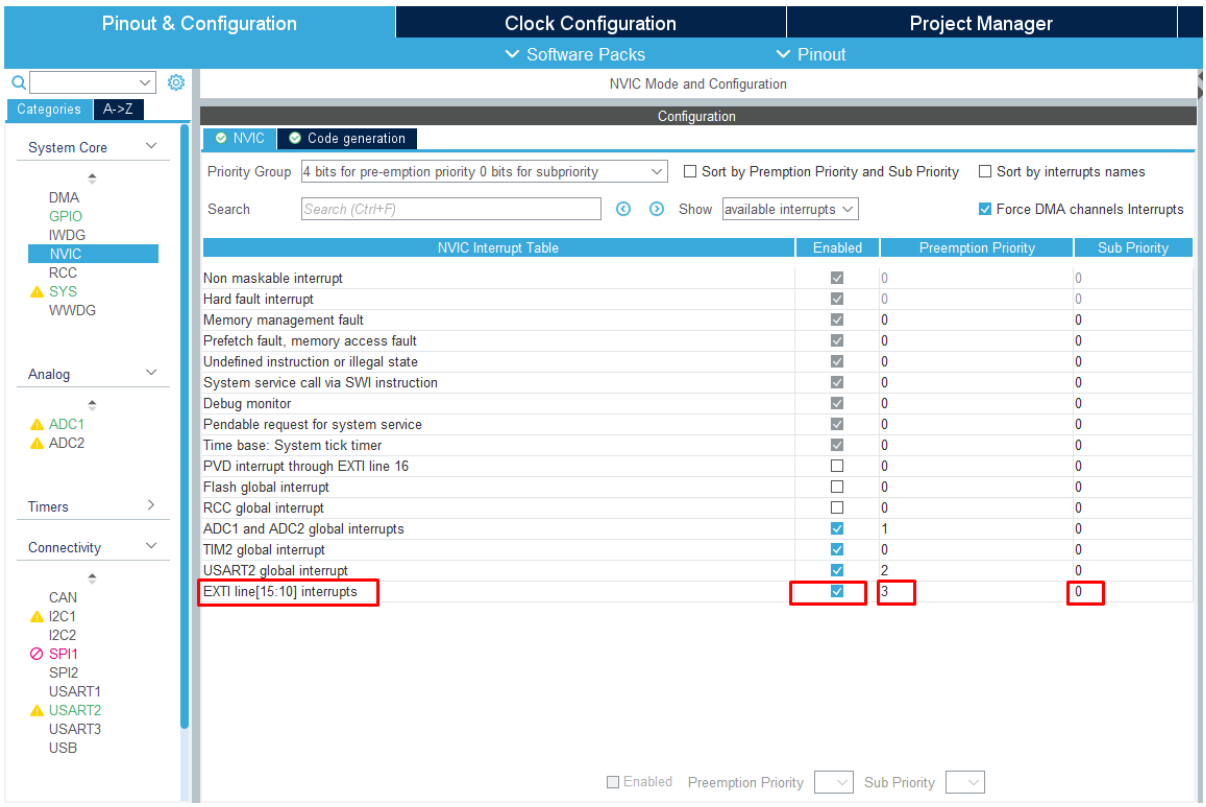
Для того, чтобы включить прерывание EXTI, необходимо в разделе Pinout & Configuration, на эскизе микроконтроллера выбрать нужный пин, и далее необходимо выставить его в EXTI так, как показано на рисунке.



Для того, чтобы включить прерывание EXTI, необходимо в разделе Pinout & Configuration, на эскизе микроконтроллера выбрать нужный пин, и далее необходимо выставить его в EXTI так, как показано на рисунке.



Если необходимо настроить приоритет прерывания, нужно выбрать раздел NVIC, и выставить Preemption Priority и Sub Priority.



Нюансы вызова HAL_Delay в прерывании

Функция HAL_Delay в микроконтроллерах STM32 использует системный таймер (SysTick) для создания задержек. Однако вызов HAL_Delay внутри прерывания может привести к нескольким проблемам:

1) Блокировка выполнения других прерываний:

Прерывания имеют высокий приоритет, и пока выполняется код прерывания, другие прерывания (с более низким приоритетом) не могут быть обработаны. Если вызвать HAL_Delay, который активно использует цикл ожидания, это может привести к длительной блокировке системы, так как прерывание не завершится, пока задержка не истечет.

2) Зависимость от SysTick:

HAL_Delay использует SysTick для отсчета времени. Если SysTick также генерирует прерывания, это может привести к конфликту или непредсказуемому поведению, так как прерывание SysTick может быть заблокировано текущим прерыванием.

3) Потеря производительности:

Использование HAL_Delay в прерывании приводит к бесполезному ожиданию, что снижает общую производительность системы. Вместо этого рекомендуется использовать более эффективные методы, такие как флаги или таймеры.

4) Риск переполнения стека:

Если прерывание вызывается часто, а HAL_Delay занимает значительное время, это может привести к накоплению вызовов и переполнению стека.

Задание

1. Реализовать внешний триггер в виде отдельного GPIO-пина STM32, логический уровень которого будет переводиться из 0 в 1, при нажатии кнопки.

2. Реализовать прерывание по внешнему триггеру GPIO-пина из шага №1. При захвате данного прерывания, реализуйте зажигание светодиода на плате.

3. Реализуйте три внешних триггера (3 EXTI-) для трёх вариаций мигания светодиода:
   - раз в 200 мс (100 мс – светодиод включен, 100 мс - светодиод выключен);
   - раз в 1000 мс (500 мс светодиод включен, 500 мс - светодиод выключен);
   - раз в 2000 мс (1000 мс светодиод включен, 1000 мс - светодиод выключен).

   Примечание: обработка трех триггеров осуществляется в одной функции *HAL_GPIO_EXTI_Callback*

4. Результат выполнения шагов 1-3 представьте в отчёте, а также к ответу на задание приложите исходный код main.c (или если имеются, другие файлы с Вашим исходным кодом).